

Lab 6 — Raster + Terrain Visualization

(EarthPy + rasterio + richdem)

Open-Source Geovisualization • GEOG 8005 / ESCI 6000 • Feb 17, 2026

Overview

In this lab you will work with a Digital Elevation Model (DEM) and create a set of terrain visualizations in Python. You will (1) load and inspect a DEM, (2) visualize elevation using multiple color ramps and value stretches, (3) compute terrain derivatives (slope, aspect, and hillshade), and (4) create a polished “DEM + hillshade” composite that improves depth perception. The emphasis is code literacy: you will run provided demo code, explain what each block does, and identify which parameters control the most important visual and analytical choices (colormap, stretch, illumination, units).

What you will produce (portfolio-friendly)

- A DEM visualization (PNG) using at least TWO different color ramps (and a clear colorbar).
- A slope raster visualization (PNG) and an aspect raster visualization (PNG).
- A hillshade visualization (PNG) with documented illumination settings (azimuth + altitude).
- A “DEM + hillshade” overlay/composite figure (PNG) that looks publication-ready.
- A short set of written answers to the code-reading questions (screenshots are fine).

Prerequisites

- A working conda environment (e.g., geoviz) with: numpy, matplotlib, rasterio, earthpy.
- Recommended: richdem (for slope + aspect) and rioarray (optional raster convenience).
- A DEM raster you can use (GeoTIFF), OR use the provided sample download below.

```
# In terminal (example)
conda activate geoviz
conda install -c conda-forge numpy matplotlib rasterio earthpy

# Recommended terrain tools
conda install -c conda-forge richdem
conda install -c conda-forge rioarray xarray

python -c "import rasterio, earthpy; print('OK')"
```

Data for this lab

Use ONE of the following options:

Option A (recommended): Use a DEM from your course project

- Use a DEM covering (or clipped to) your study area (GeoTIFF is easiest).
- You should know (or look up) the DEM's horizontal units (meters/feet) and vertical units (elevation units).

Option B: Use a sample DEM bundled with EarthPy (fastest)

- EarthPy can download a small example DEM used in its documentation gallery.
- This option is ideal if you don't have project DEM data yet.

```
import os
import earthpy as et

# Downloads sample data to your local earth-analytics directory
data_dir = et.data.get_data("vignette-elevation")

# The example DEM file (pre_DTM.tif) lives inside that folder
dem_path = os.path.join(data_dir, "pre_DTM.tif")
print(dem_path)
```

Option C (optional): Download SRTM data for a bounding box (internet required)

If you want a DEM for a specific place, you can download SRTM tiles and clip them with the elevation package. This is optional because it depends on an internet connection and may take a few minutes.

```
# Optional: download & clip SRTM for a bounding box
# Requires: pip install elevation

import elevation

# Example bounds: (min_lon, min_lat, max_lon, max_lat)
bounds = (-81.0, 35.0, -80.5, 35.5) # <-- change for your area
out_dem = "data/raw/srtm_clip.tif"

elevation.clip(bounds=bounds, output=out_dem, product="SRTM1")
print("Wrote:", out_dem)
```

Submission (screenshots only)

Submit a set of image files containing these screenshots:

1. Successful imports + package versions (rasterio, earthpy, richdem if used).
2. Raster inspection output: CRS, shape, resolution, nodata value, min/max (after masking).
3. DEM plot with at least TWO colormaps (show your chosen vmin/vmax or percentile stretch).
4. Slope plot + aspect plot (include what units you used: degrees vs percent).
5. Hillshade plot (include azimuth + altitude).
6. DEM + hillshade composite plot (and the alpha / blending choice).
7. Your answers to the code-reading questions (screenshots of a markdown file, notebook, or terminal are fine).

Part 1 — Setup + DEM loading

1.1 Import + environment check

- Import core packages and print versions (this helps debugging and reproducibility).
- If an import fails, fix your environment before continuing.

```
import os
import numpy as np
import matplotlib
import matplotlib.pyplot as plt

import rasterio as rio
import earthpy as et
import earthpy.plot as ep
import earthpy.spatial as es

print("numpy:", np.__version__)
print("matplotlib:", matplotlib.__version__)
print("rasterio:", rio.__version__)
print("earthpy:", et.__version__)

# Optional (recommended)
try:
    import richdem as rd
    print("richdem:", rd.__version__)
except Exception as e:
    print("richdem: NOT INSTALLED (ok) ->", e)
```

1.2 Open and inspect the DEM

- Read the DEM as a NumPy array.
- Inspect metadata (CRS, resolution, nodata).
- Mask nodata / invalid values as np.nan so plots behave well.

```
DEM_PATH = "data/raw/dem.tif" # <-- change (or set from Option B/C)

with rio.open(DEM_PATH) as src:
    dem = src.read(1).astype("float32")
    profile = src.profile
    crs = src.crs
    transform = src.transform
    nodata = src.nodata

print("CRS:", crs)
print("Shape (rows, cols):", dem.shape)
print("Resolution:", (transform.a, -transform.e))
print("Nodata:", nodata)

# Mask nodata (and any other invalid values you decide)
if nodata is not None:
```

```
dem[dem == nodata] = np.nan

print("Min/Max (masked):", np.nanmin(dem), np.nanmax(dem))
```

Check your understanding (Part 1)

8. What is a DEM, and what does each pixel value represent?
9. Why do we convert nodata values to np.nan before plotting?
10. What does “resolution” mean for a raster, and how do you read it from the rasterio transform?

Part 2 — Visualize elevation (color ramps + value stretch)

2.1 Plot the DEM with different colormaps

Try at least two different color ramps. For elevation, common choices include 'terrain', 'gist_earth', and perceptually-uniform ramps like 'viridis' or 'cividis'.

```
# Simple DEM plot (EarthPy helper)
ep.plot_bands(
    dem,
    cmap="terrain",  # try: "gist_earth", "viridis", "cividis"
    title="DEM (Elevation)",
    figsize=(10, 6),
)
plt.show()
```

2.2 Apply a percentile stretch (recommended for prettier maps)

Real DEMs often have outliers (spikes, pits). A percentile stretch can improve contrast by clipping extreme values.

```
# Percentile stretch (2nd-98th is a common starting point)
vmin, vmax = np.nanpercentile(dem, (2, 98))

ep.plot_bands(
    dem,
    cmap="terrain",
    vmin=vmin,
    vmax=vmax,
    title=f"DEM (2-98% stretch): vmin={vmin:.1f}, vmax={vmax:.1f}",
    figsize=(10, 6),
)
plt.show()
```

2.3 (Optional) Plot a histogram to justify your stretch

```
vals = dem[np.isfinite(dem)]
```

```
plt.figure(figsize=(8, 4))
plt.hist(vals, bins=60)
plt.title("Elevation histogram (masked)")
plt.xlabel("Elevation")
plt.ylabel("Pixel count")
plt.show()
```

Check your understanding (Part 2)

11. Which parameter changes the color ramp? Give two valid cmap names.
12. What do vmin and vmax do, and why might a percentile stretch be preferable to min/max?
13. If you change the stretch, does it change the underlying data values? Why or why not?

Part 3 — Slope + aspect (terrain derivatives)

Slope and aspect are derived from elevation differences between neighboring pixels. In this lab, we recommend using `richdem` because it provides reliable terrain derivatives with concise code.

3.1 Compute slope and aspect with `richdem` (recommended)

```
# Requires: conda install -c conda-forge richdem
import richdem as rd

# Load DEM with georeferencing (best practice)
dem_rd = rd.LoadGDAL(DEM_PATH)

# Slope options:
# - 'slope_degrees' (0-90) is easy to interpret
# - 'slope_riserun' (rise/run) can be converted to percent slope
slope_deg = rd.TerrainAttribute(dem_rd, attrib="slope_degrees")

# Aspect is a circular direction (0-360 degrees, often 0/360 = North)
aspect = rd.TerrainAttribute(dem_rd, attrib="aspect")
```

3.2 Plot slope and aspect

```
# Slope
ep.plot_bands(
    np.array(slope_deg),
    cmap="magma",
    title="Slope (degrees)",
    figsize=(10, 6),
)
plt.show()

# Aspect (use a cyclic colormap if available)
ep.plot_bands(
    np.array(aspect),
    cmap="twilight", # try: "hsv" if twilight is not available
```

```

    title="Aspect (degrees, cyclic)",
    figsize=(10, 6),
)
plt.show()

```

Check your understanding (Part 3)

14. In one sentence: what does slope measure, and what does aspect measure?
15. Why is aspect considered a circular variable?
16. If your DEM is in degrees (lat/lon), what problem can that create for slope/aspect calculations?

Part 4 — Hillshade (illumination + shading)

A hillshade is a shaded-relief representation of the surface. You can adjust the illumination source using azimuth (direction of the light) and altitude (sun angle above the horizon).

4.1 Create hillshade with EarthPy

```

# EarthPy hillshade from the DEM array
# azimuth: 0=N, 90=E, 180=S, 270=W (default often ~315)
# altitude: 0-90 (higher = less shadow)
azimuth = 315
altitude = 45

hill = es.hillshade(dem, azimuth=azimuth, altitude=altitude)

ep.plot_bands(
    hill,
    cmap="Greys",
    cbar=False,
    title=f"Hillshade (azimuth={azimuth}, altitude={altitude})",
    figsize=(10, 6),
)
plt.show()

```

4.2 (Optional) Compare multiple hillshades

```

# Compare 4 lighting settings (small multiples)
settings = [(315, 45), (45, 45), (315, 25), (315, 70)]

fig, axes = plt.subplots(2, 2, figsize=(12, 9))
axes = axes.ravel()

for ax, (az, alt) in zip(axes, settings):
    hs = es.hillshade(dem, azimuth=az, altitude=alt)
    ax.imshow(hs, cmap="Greys")
    ax.set_title(f"az={az}, alt={alt}")
    ax.set_axis_off()

```

```
plt.tight_layout()
plt.show()
```

Check your understanding (Part 4)

17. What is the visual purpose of hillshade in cartography?
18. Which parameter changes the direction of light? Which changes the sun height?
19. Why might you choose a lower altitude (more shadows) vs a higher altitude (flatter look)?

Part 5 — DEM + hillshade composite (polished terrain map)

A common terrain design is to draw elevation with a color ramp and overlay a semi-transparent hillshade. This keeps the quantitative color meaning while adding 3D structure.

5.1 Overlay hillshade on top of colored DEM

```
# 1) Plot the DEM as the base (color)
fig, ax = plt.subplots(figsize=(10, 6))
ep.plot_bands(
    dem,
    ax=ax,
    cmap="terrain",
    title="DEM + hillshade overlay",
    cbar=True,
)

# 2) Overlay hillshade (grayscale) with transparency
ax.imshow(hill, cmap="Greys", alpha=0.35) # try alpha: 0.2 -> 0.6
ax.set_axis_off()

plt.show()
```

5.2 (Optional) Export the figure

```
# Save at high resolution for reports / portfolio
out_png = "outputs/dem_hillshade.png"
os.makedirs("outputs", exist_ok=True)

fig.savefig(out_png, dpi=300, bbox_inches="tight")
print("Saved:", out_png)
```

Check your understanding (Part 5)

20. Which line controls the transparency of the hillshade overlay?
21. If the overlay makes the map too dark, what two things can you adjust?
22. Why is it helpful to keep a colorbar on the DEM even when adding hillshade?

Part 6 — (Optional) Write derived rasters to GeoTIFF

If you want to reuse your derived layers (slope, aspect, hillshade) in GIS or future notebooks, export them as GeoTIFFs using the original DEM profile.

```
# Write GeoTIFFs with rasterio (keeps CRS, transform, etc.)
os.makedirs("outputs", exist_ok=True)

def write_tif(path, arr, profile):
    prof = profile.copy()
    prof.update(dtype="float32", count=1, nodata=np.nan)
    with rio.open(path, "w", **prof) as dst:
        dst.write(arr.astype("float32"), 1)

write_tif("outputs/slope_degrees.tif", np.array(slope_deg), profile)
write_tif("outputs/aspect_degrees.tif", np.array(aspect), profile)
write_tif("outputs/hillshade.tif", hill.astype("float32"), profile)

print("Wrote GeoTIFFs to outputs/")
```

Helpful references (for further learning)

- EarthPy hillshade vignette (DEM + hillshade overlay):
https://earthpy.readthedocs.io/en/latest/gallery_vignettes/plot_dem_hillshade.html
- Earth Data Science (Earth Lab): slope + aspect with richdem:
<https://earthdatascience.org/tutorials/get-slope-aspect-from-digital-elevation-model/>
- NEON Learning Hub: create a hillshade in Python: <https://www.neonscience.org/resources/learning-hub/tutorials/create-hillshade-py>
- Rasterio docs (read/write GeoTIFFs, profiles, metadata): <https://rasterio.readthedocs.io/>
- Matplotlib colormap reference (choose better ramps):
https://matplotlib.org/stable/gallery/color/colormap_reference.html

Tip: In your README, cite where a code pattern came from (source name + URL).